

— Data Quality Plan — (DQP)

Core Objectives

1. Read the raw training dataset `ppr-group-25208508-train.csv`.
 2. Apply rule-based cleaning (types, missing values, duplicates, outliers, and business rules).
 3. Produce:
 - `ppr-group-25208508-preview.csv` (cleaned output in this notebook)
 - a step-by-step processing log for auditability.
-

1) Imports & Settings

Import required libraries and configure pandas display options for readable outputs.

2) Load Dataset

Load the training dataset `ppr-group-25208508-train.csv`.

3) Quick Structure Check (Before Cleaning)

Report column names, missingness, dtypes, and duplicate-row count as the pre-cleaning baseline.

4) Utility Functions

This section defines reusable helpers for text normalization, price parsing, address feature extraction, and process logging.

Function summary: – `DQLog`: records per-step row/column changes and notes for an auditable cleaning trail. – `clean_whitespace(x)`: trims text and normalizes repeated spaces. – `parse_price_to_numeric(series)`: converts price strings (e.g., €123,456.78) to numeric values. – `extract_dublin_district(addr)`: extracts Dublin district patterns (e.g., Dublin 4, D6W). – `extract_area_token(addr)`: derives a coarse location token from address text without external APIs.

These helpers improve consistency, explainability, and reproducibility in downstream steps.

5) Start Cleaning — Work on a Copy

Use a working copy (`df`) instead of modifying `df_raw` directly.

6) Basic Standardization

- Normalize whitespace in text fields.
 - Parse date fields.
 - Convert Price (€) to numeric.
 - Create time-derived features (Sale Year, Sale Month) to align with DQR.
-

7) Drop Fully Empty / Constant Columns (If Any)

Detect and remove fully empty columns or constant columns, then log the decision.

8) Duplicate Rows

Remove exact duplicate rows and record the number removed (currently 9 rows).

9) Core Field Validations

Apply minimum validity checks: – Price (€) must be positive. – Sale date must be valid/parseable. – County must be non-empty.

10) Address Handling — Feature Engineering then Drop Raw Address

Background and Problem

Address is a high-cardinality free-text field with inconsistent formatting and high sparsity when encoded directly. DQR also identified high missingness in Eircode, so raw location fields are not reliable as-is.

Strategy (Extract First, Drop Later)

1. Engineer compact, explainable location/morphology features from Address.
2. Drop raw Address after feature extraction to reduce noise, dimensionality, and privacy risk.

Why Function-Based Feature Construction

- **Consistency:** standardizes text and parsing logic.
- **Explainability:** engineered features are interpretable.
- **Reproducibility:** deterministic rules, minimal external dependency.
- **Auditability:** each transformation is traceable in the processing log.

Relation to Address_to_GPS Pipeline

The Address_to_GPS/ workflow (geocoding -> district extraction -> rematch -> map validation) was used as supporting exploration to validate geographic signal extraction. For the main DQP pipeline, we keep an API-light, reproducible approach while preserving the same feature-engineering intent.

Advantages

- Reduces dimensionality and overfitting risk.
- Improves generalization by reducing raw-text noise.
- Preserves useful geographic signal via engineered features.
- Keeps the pipeline reproducible and report-ready.

Final Decision

Use Address only as an intermediate source for feature generation, then remove the raw column.

10.1) Address-Derived Feature Distribution and area_token Cardinality Reduction (N >= 50)

This step has two goals: 1. Summarize distributions of address-derived features (is_apartment, has_number, dublin_district, area_token) to verify reasonableness and interpretability. 2. Reduce high cardinality in area_token using frequency thresholding: - Keep tokens with frequency >= 50. - Merge lower-frequency tokens into **Other**.

This retains major location signal while controlling feature explosion and improving model stability.

11) VAT Rule

For rows where Description of Property == 'New Dwelling house /Apartment' and VAT Exclusive == Yes, adjust price using VAT (13.5%).

Feature handling: - Keep original Price (€). - Create Price_Adjusted_VAT. - Create vat_adjusted_flag to mark adjusted rows.

12) Clamping after VAT Adjustment

Price_Adjusted_VAT_Clamped is used as the standardized target for downstream analysis/modeling because it addresses two issues: - inconsistent price definition (VAT-inclusive vs VAT-exclusive), - heavy-tailed distribution and extreme outliers.

Applied rule: - compute the 5th and 95th percentiles on Price_Adjusted_VAT, - clip values outside this range, - keep original/intermediate columns for auditability.

In DQR, multiple clamping levels can still be compared for exploratory analysis, but this notebook sets the final DQP cleaning rule.

13) High Missingness Columns — Keep vs Drop

Based on DQR findings (e.g., Property Size Description, Eircode), we apply a conservative missing-data policy: – If missingness is extremely high and analytical value is low: drop. – If potentially useful: keep with missing flags and appropriate imputation.

A configurable threshold is used: DROP_MISSING_PCT = 90.0.

13.1) Explicit Eircode Drop

Eircode is explicitly removed and logged due to high missingness and limited utility as a stable modeling feature in this pipeline.

14) Fill Missing Values (Conservative, Aligned with Downstream Modeling)

DQP handling in this notebook (training data)

To produce a modeling-ready cleaned dataset: – categorical fields -> fill with Unknown – numeric fields -> fill with median

This matches the cleaning logic used to generate train_data_clean.csv in model_training/linear_regression.ipynb.

Connection to downstream test-set handling

In model evaluation (test_data.csv), the implemented flow mainly uses rule-based cleaning plus invalid-row filtering (text normalization, parsing, feature construction, then filtering). The test path is therefore not a full symmetric all-field Unknown/median imputation pipeline.

Why this design

- Training set: enforce a stable, reproducible no-NaN dataset for model fitting.
 - Test set: keep preprocessing closer to realistic inference conditions with minimal extra assumptions.
-

15) Post-Clean Validation

Checks include: – whether missing values remain, – whether the price column is strictly positive, – whether key assumptions hold after cleaning.

16) DQ Processing Log

Summarize row/column impact for each cleaning step. The log serves as an auditable evidence trail. A more complete DQLog usage is also implemented in model_training/linear_regression.ipynb.

17) Export Clean Dataset

Export the final cleaned dataset to ppr-group-25208508-preview.csv.

18) Quick Peek of Output

Sample a few rows to verify the exported dataset format and values.

19) Appendix

[Click here to check the original code and result](#)